

Portfolio Intelligence Dashboard

Foundation Workflow Map · Week 1 — Foundation

Author: Folahan Williams — Investment Analyst Intern | **Team:** Funds Management & Technology | **Date:** Thursday, 18 June

Purpose. Show, on one page, how the system is wired end to end — how data flows, where it is fetched, what the backend computes, what the frontend renders, and what the Portfolio Manager actually sees on screen.

1 One-line picture

A fund's **holdings, prices, and benchmark** go in. A single screen comes out showing **what we hold, how much risk we carry, and what drove performance** — computed once on the server, rendered instantly on the client.

The frontend never computes fund math. The backend never handles presentation. This separation is the load-bearing decision: the analytics layer is built so it can graduate to production use without a rewrite.

2 The workflow at a glance

1 · DATA SOURCE (mock files — no database)

holdings.csv (shares, sector, region, currency) · prices.csv (daily history) · benchmark.csv (index weights & returns) · fx.csv (rates → USD)



2 · BACKEND — Python · FastAPI · pandas / numpy

Data Loader (validate · skip & flag bad rows) → Analytics Engine (FX-normalise → all fund math) → REST Endpoints (/summary · /exposure · /risk · /attribution)



3 · FRONTEND — React · TypeScript · Vite · Tailwind · Recharts/Plotly

API client (fetch JSON) → Dashboard shell (tabs · window · filters) → Charts & tables (pure presentation, no fund math)



4 · PORTFOLIO MANAGER

One screen: positioning · risk · attribution. Change window / filter loops back to the backend, recomputes < 500 ms.

Read it top to bottom: mock files → backend loads & validates → analytics engine computes all fund math → thin REST endpoints serve JSON → frontend fetches and renders → manager sees one screen. Changing the date window or a filter loops the request back to the backend, and the screen recomputes in under half a second.

3 Where the data is fetched

For the prototype, **all data is local mock files** loaded by the backend — no database, no external dependencies, runs on a laptop or the internal server.

Input	Holds	Used for
holdings.csv	shares, sector, region, native currency per security	AUM, weights, exposure
prices.csv	daily price history (November last year onward)	returns, volatility, beta, VaR, drawdown

Input	Holds	Used for
benchmark.csv	index sector/region weights & returns (S&P tech basket)	active weights, Brinson attribution
fx.csv	currency rates to USD	currency-normalising every cross-holding total

Currency: every holding is normalised to **USD** before any total is aggregated — once, inside the backend, before metrics are computed.

Production path (out of scope now, designed for later): the loader is the *only* place that touches raw data. Swapping mock CSVs for live custodian / market-data feeds (e.g. Finnhub) is a change to that one layer — the analytics engine and frontend are untouched.

4 The backend — where the thinking happens

Stack: Python · FastAPI · pandas / numpy. Three stages, in order:

1. Load & validate — read the mock files, check each row. Malformed or missing rows are **skipped and flagged**, never crash the app. **2. Analytics engine** — FX-normalise, then compute every metric (AUM, weights, active weights, volatility, beta, Sharpe, VaR 95/99, max drawdown, contribution, Brinson allocation vs selection), each validated against an independent reference to within $\pm 0.1\%$. **3. Serve** — four thin REST endpoints return ready-to-render JSON.

Endpoint	Powers the screen section	Returns
GET /summary	Portfolio Summary	AUM, P&L, total & active return, top contributors/detractors
GET /exposure	Sector & Geographic Exposure	sector/region weights, signed active weights, HHI / effective N, heatmap data
GET /risk	Risk Metrics	volatility, beta, Sharpe, VaR (95/99, historical & parametric),

Endpoint	Powers the screen section	Returns
		correlation matrix, max drawdown
GET /attribution	Performance Attribution	MTD/QTD/YTD/1Y returns, contribution by security & sector, Brinson alloc vs selection

5 The frontend — what gets rendered

Stack: React · TypeScript · Vite · Tailwind · Recharts / Plotly. A **pure presentation layer**: it fetches JSON from the four endpoints and renders it, holding no fund math — only display state (open tab, date window, filters). Each section maps one-to-one to one backend endpoint, so the data path for any view is obvious and traceable.

Portfolio Summary /summary	Sector & Geo Exposure /exposure	Risk Metrics /risk	Performance Attribution /attribution
-------------------------------	------------------------------------	-----------------------	---

6 What the user sees (UI / UX)

The Portfolio Manager opens **one screen** and gets the three core answers in seconds — no scrolling needed for the primary insights.

+-----+			
PORTFOLIO INTELLIGENCE		[Window: 1M v] [Filter: All v]	
+-----+			
AUM (USD)		Total Return	
\$XX.Xm		+X.X%	
Top contributors: ^ AAPL ^ MSFT		Active vs Benchmark	
HHI / effective N		Top detractors: v NVDA v AMD	
+-----+			
SECTOR & GEOGRAPHIC EXPOSURE		RISK METRICS	
[exposure heatmap]		Volatility Beta Sharpe	
active weights (signed vs bench)		VaR 95%/99% Max Drawdown	
HHI / effective N		correlation heatmap	
+-----+			
PERFORMANCE ATTRIBUTION			
MTD QTD YTD 1Y		contribution by sector/security	
Brinson: allocation vs selection by sector			



Interaction model: the manager changes the **window** (MTD / QTD / YTD / 1Y) or a **filter** (sector / region) once, at the top. Every panel below **recomputes from the backend in under 500 ms** — positioning, risk and attribution update together against the same window. **Empty and error states are explicit:** missing or flagged data is stated clearly, never shown as a blank or a wrong number.

What the manager experiences	Target
Time to the three core answers	< 30 s from load
Initial load (mock data)	< 2 s
Recompute on window/filter change	< 500 ms
Metric accuracy vs independent reference	within $\pm 0.1\%$

7 Why it's built this way (the foundation point)

A 3-week prototype on mock data, but built as a **production-track foundation**:

Decision	Why it matters for production
Analytics separated from presentation	the fund math lifts into production unchanged
One loader owns all data access	mock CSVs swap for live feeds in a single place
Thin, typed REST contract	frontend and backend evolve independently
Graceful degradation + $\pm 0.1\%$ validation	trustworthy enough to graduate to formal use without a rewrite

8 Week 1 status (Foundation)

Item	State
Stack & data confirmed	Done
Workflow / foundation map	Done (this document)
Data pipeline (load → validate → normalise)	In progress
Dashboard shell	In progress
Portfolio Summary working end to end	Next

Next: stand up the pipeline + shell, deliver the working Portfolio Summary, and begin AI sector research — leading into Week 2 (Exposure, Risk, Attribution).